

1주 차 보고서

(2026년 4 월 5 일 ~ 2026년 4 월 11 일)

작성자

박신우

이번 주 회의 내용

- 중간발표까지 목표 정리
- 서버 캐릭터 이동 확인
- 아이템,캐릭터 스텟 정리

이번 주 한 일

4.7 언리얼 시네마틱 공부

1스테이지에서 2스테이지로 넘어갈 때 시네마틱 같은 것이 있으면 좋겠다는 생각이 들어서, 시네마틱에 관련된 지식이 없어, 자료를 찾아보며 공부를 하는 시간은 가졌다.

4.8 회의 진행

회의 때 시네마틱은 중간발표때까지 완성하기 힘들 것 같다는 결론이 나와서, 미루어 두기로 하였다.

4.9~10 트럭으로 좀비 치기.

좀비를 트럭으로 쳤을 때 좀비가 날아가게 하는 함수를 만들어 주었다.

OnTruckMeshHit() 함수에서 좀비가 날아가게 하는 함수를 호출하게 만들었다.

```
void ATruck::OnTruckMeshHit(UPrimitiveComponent* HitComponent, AActor* OtherActor, UPrimitiveComponent* OtherComp, FVector NormalImpulse, const FHitResult& Hit)
{
    // 유효한 액터인지 검사
    if (!OtherActor || OtherActor == this)
    {
        return;
    }
    // 좀비가 아니거나 죽었으면 무시
    ABaseZombie* Zombie = Cast<ABaseZombie>(OtherActor);
    if (!Zombie || !Zombie->IsAlive())
    {
        return;
    }
    ProcessZombieImpact(Zombie, Hit.ImpactPoint, GetVelocity().GetSafeNormal(), GetVelocity().Size());
}
```

OnTruckMeshHit()함수는 시작할 때 메시에 OnComponentHit를 바인딩해서 실제 물리 충돌을 받도록 하였다.

```
void ATruck::BeginPlay()
{
    Super::BeginPlay();

    if (USkeletalMeshComponent* TruckMesh = GetMesh())
    {
        TruckMesh->SetNotifyRigidBodyCollision(true);
        TruckMesh->OnComponentHit.AddDynamic(this, &ATruck::OnTruckMeshHit);
    }
}
```

우선적으로, 매 프레임마다 검사를 해버리면 좀비를 여러 번 치면서 버그가 일어나기 때문에, 좀비의 충돌시간을 업데이트 하여 좀비를 여러 번 치는 것을 방지해 주었다.

```
void ATruck::ProcessZombieImpact(ABaseZombie* Zombie, const FVector& ImpactPoint, const FVector& ImpactDirection, float ImpactSpeed)
{
    // 속도가 느리거나 좀비가 죽으면 무시
    if (!Zombie || !Zombie->IsAlive() || ImpactSpeed < ZombieImpactMinSpeed)
    {
        return;
    }

    // 좀비를 여러 번 쳐버리는 것을 방지
    const float CurrentTime = GetWorld()->GetTimeSeconds();
    if (const float* LastImpactTime = LastZombieImpactTimes.Find(Zombie))
    {
        if (CurrentTime - *LastImpactTime < ZombieImpactCooldown)
        {
            return;
        }
    }

    // LastZombieImpactTimes 맵에 좀비의 마지막 충돌 시간을 업데이트
    LastZombieImpactTimes.Add(Zombie, CurrentTime);
}
```

이번 주 한 일

그 후 방향 계산에 실패해도 좀비를 앞으로 날릴 수 있도록 방향을 보정해준 후, 좀비가 입을 데미지와 좀비를 날아가게 하는 넉백을 속도에 따라 계산해주었다.

```
// 전달받은 충돌 방향이 거의 0 이면 트럭 전방으로 간주
const FVector SafeImpactDirection = ImpactDirection.IsNearlyZero() ? GetActorForwardVector() : ImpactDirection.GetSafeNormal();

// 좀비가 받을 데미지 속도에 따라 계산.
const float Damage = FMath::GetMappedRangeValueClamped(
    FVector2D(ZombieImpactMinSpeed, ZombieImpactFatalSpeed),
    FVector2D(ZombieImpactMinDamage, ZombieImpactMaxDamage),
    ImpactSpeed);

// 좀비의 넉백을 속도에 따라 조절
const float KnockbackScale = FMath::GetMappedRangeValueClamped(
    FVector2D(ZombieImpactMinSpeed, ZombieImpactFatalSpeed),
    FVector2D(1.0f, 1.6f),
    ImpactSpeed);
```

이 함수에서 사용한 `GetMappedRangeValueClamped` 함수는 `InputRange` 범위에 대하여, `OutputRange` 백분율로 `value`를 리턴해주는 함수이다.

```
template<typename T, typename T2>
[[nodiscard]] static FORCEINLINE auto GetMappedRangeValueClamped(const UE::Math::TVector2<T>& InputRange, const UE::Math::TVector2<T>& OutputRange, const T2 Value)
{
    using RangePctType = decltype(T() * T2());
    const RangePctType ClampedPct = Clamp<RangePctType>(GetRangePct(InputRange, Value), 0.f, 1.f);
    return GetRangeValue(OutputRange, ClampedPct);
}
```

그 후 좀비의 위치를 가져와 트럭의 로컬 좌표로 변경을 해주고, 좀비가 트럭의 몸체에 부딪혔는지 검사를 해준다.

애초에 `OnTruckMeshHit()`에서 이 함수를 호출 해주는데, 검사를 해주는 이유는 `OnTruckMeshHit()`는

1. 좀비가 트럭에 스친 거 일 수 있어서.
2. 후에 나올 즉사조건을 더 자연스럽게 만들기위해서.

이렇게 2가지 이유 때문이다.

```
// 좀비의 위치를 트럭의 로컬 좌표로 변경
const FVector LocalZombieLocation = GetActorTransform().InverseTransformPosition(Zombie->GetActorLocation());
// 트럭 메쉬 크기를 가져옴
const FVector MeshExtent = GetMesh() ? GetMesh()->Bounds.BoxExtent : FVector(150.0f, 100.0f, 100.0f);
// 좀비가 트럭 몸체 범위 안에 있는지 검사
const bool bWithinTruckLength =
    LocalZombieLocation.X > -MeshExtent.X * 1.15f &&
    LocalZombieLocation.X < MeshExtent.X * 1.15f;
const bool bWithinTruckWidth = FMath::Abs(LocalZombieLocation.Y) < MeshExtent.Y * 1.35f;
const bool bWithinTruckHeight = FMath::Abs(LocalZombieLocation.Z) < MeshExtent.Z * 1.5f;
const bool bTruckBodyImpact = bWithinTruckLength && bWithinTruckWidth && bWithinTruckHeight;
```

그 후 좀비에게 데미지를 주고, 특정 속도 이상이면 즉사를 하도록 조건을 만들었다.

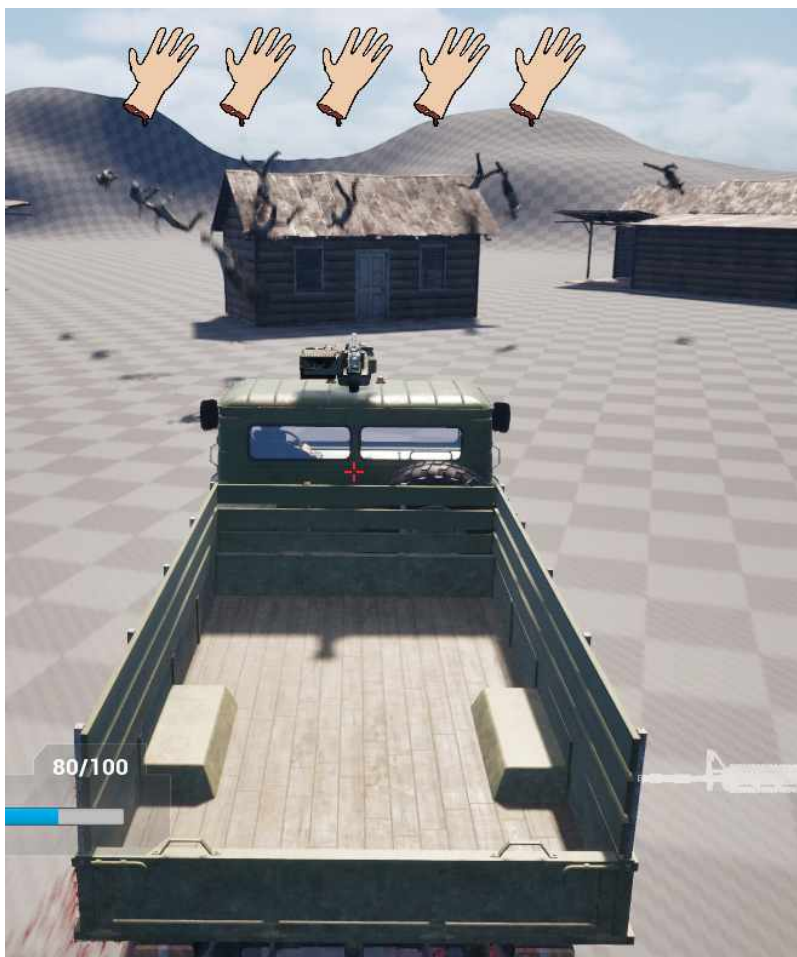
```
// 좀비에게 데미지를 줌
UGameplayStatics::ApplyPointDamage(
    Zombie,
    Damage,
    ImpactFlingDirection,
    DamageHit,
    GetController(),
    this,
    nullptr);
// 좀비 즉사 조건
const bool bCheatFlingImpact = (bTruckBodyImpact) && ImpactSpeed >= ZombieImpactMinSpeed;
```

이번 주 한 일

이제 죽사 조건이 성립되었다면 바로 죽이고,
살아있다면 캐릭터를 LaunchCharacter를 활용해 넉백 시키고,
죽은 경우에 레그돌을 하여 임펄스를 가해 좀비를 날려 버렸다.

```
if (Zombie->IsAlive() &&  
    (ImpactSpeed >= ZombieImpactFatalSpeed ||  
     bCheatFlingImpact))  
{  
    Zombie->Die();  
}  
  
// 살아있다면 캐릭터를 넉백  
if (Zombie->IsAlive())  
{  
    const FVector LaunchVelocity =  
        ImpactFlingDirection * (ZombieImpactKnockback * KnockbackScale * 1.2f) +  
        FVector::UpVector * (ZombieImpactUpwardKnockback * KnockbackScale);  
    Zombie->LaunchCharacter(LaunchVelocity, true, true);  
    return;  
}  
  
// 죽은 경우 레그돌 하여 넉백  
if (USkeletalMeshComponent* ZombieMesh = Zombie->GetMesh())  
{  
    const FVector WorldImpulse =  
        ImpactFlingDirection * (ZombieImpactImpulse * KnockbackScale * 1.2f) +  
        FVector::UpVector * (ZombieImpactImpulse * 0.2f * KnockbackScale);  
    ZombieMesh->AddImpulseAtLocation(WorldImpulse, ImpactPoint, FName(TEXT("pelvis")));  
}
```

시원하게 날리는 모습을 확인 할 수 있었다.



다음 주 할 일

- 좀비 행동트리
- 좀비 애니메이션 동기화

특이사항

시험공부 때문에 많이 진행하지 못하였다.